

Introduction to Distributed Data Processing

Distributed data processing has transformed the way organisations handle large-scale analytics. By splitting work across many nodes, frameworks such as Apache Spark can process terabytes of data in minutes that would take hours on a single machine.

This article introduces the core concepts: resilient distributed datasets (RDDs), DataFrames, and the Catalyst query optimiser, and explains how they work together to deliver high-throughput, fault-tolerant computation.

Apache Spark Architecture

A Spark application consists of a driver process and a set of executor processes. The driver coordinates the job by constructing a directed acyclic graph (DAG) of stages, while executors perform the actual data transformations.

The cluster manager (YARN, Kubernetes, or Spark's built-in standalone mode) allocates resources. Executors cache intermediate data in memory, which is the key reason Spark outperforms Hadoop MapReduce for iterative workloads.

DataFrames and the Catalyst Optimiser

The DataFrame API provides a schema-aware abstraction over RDDs. Queries are expressed using high-level operations (filter, groupBy, join) that Catalyst compiles into an optimised physical plan.

Catalyst applies rule-based and cost-based optimisations: predicate push-down moves filters as close to the data source as possible; column pruning removes unused fields before reading; and join reordering minimises intermediate data sizes.

Conclusion and Further Reading

Apache Spark continues to evolve rapidly. Spark 3.x introduced Adaptive Query Execution (AQE), which re-optimises query plans at runtime based on actual partition statistics, dramatically reducing skew and over-partitioning.

For further reading consult the official documentation at spark.apache.org, or explore the Learning Spark book (2nd edition, O'Reilly) for end-to-end production examples.